Bhartiya Vidya Bhavan's, Sardar Patel Institute of Technology
(An Autonomous Institute Affiliated to University of Mumbai)
Bhavan's Campus, Munshi Nagar, Andheri
(West), Mumbai-400058-India)

Name	Hetanshi Shah
UID no.	2023300199
Exp No.	7
Problem Statement	To expose the existing healthcare Retrieval-Augmented Generation (RAG) application through a FastAPI endpoint, with input validation, controlled error handling, basic logging, and response-time measurement.

Github: https://github.com/hetanshi18/llm-production-rag-api

1. Theory

A Retrieval-Augmented Generation (RAG) application combines information retrieval with a language model. When a user submits a question, the retriever searches a knowledge base for relevant text chunks. Those chunks are added to a prompt, and the language model generates an answer grounded in the retrieved context. This can reduce unsupported answers, but it does not guarantee that every generated answer is correct.

FastAPI exposes Python functionality through HTTP endpoints so that other applications can send requests and receive structured responses. Pydantic request validation checks that incoming data meets requirements before the request is processed. Logging records operational events, while response-time measurement records how long a request takes.

This implementation uses Ollama as a local LLM backend. The language model runs on the same computer and is called through Ollama's local HTTP API. The embedding model converts text into vectors, and Chroma stores and searches those vectors. The embedding model and vector store are initialized once when the RAG module is imported, rather than rebuilt for every request.

The /health endpoint reports whether the FastAPI process is running; it does not check whether Ollama is available. The /ask endpoint accepts a question, retrieves relevant context, calls the LLM when the retrieval distance is within the configured threshold, and returns the answer, source references, and response time. A distance threshold is a heuristic and must be tested with supported and unsupported questions.

Retrieval-Augmented Generation (RAG): RAG is a pattern in which an application retrieves relevant information from a defined collection of documents before asking a language model to generate a response. It is useful when answers should be based on a specific knowledge base rather than only on information encoded in the model during training. In this experiment, the knowledge base consists of a diabetes PDF, a dengue DOCX file, and a mental-health text file.

Local LLM backend: Ollama serves the selected qwen2.5:0.5b language model through a local HTTP endpoint. The application sends the prompt using an HTTP POST request, disables streaming, sets the temperature to zero, limits the generated output to 180 tokens, and applies a timeout. A local model can keep inference on the computer, but its speed and memory requirements depend on the model and hardware.

API validation and HTTP status codes: The AskRequest Pydantic model validates the incoming JSON. It trims leading and trailing whitespace, rejects a blank question, and limits the length to 1,000 characters. FastAPI returns

HTTP 422 for invalid request data. The application maps LLM backend failures to HTTP 502 and unexpected application errors to HTTP 500, while returning generic messages to the client instead of internal stack traces.

Logging, latency, and health checks: The application measures elapsed time with time.perf_counter() and returns response_time_ms for successful /ask requests. Logs record successful requests, failures, and latency while deliberately avoiding logging the question text. GET /health returns a simple status indicating that the API process is running; it does not verify that the Ollama service or the retrieval database is healthy.

Persistent vector store and initialization: The embedding model and Chroma store are initialized when support_rag.py is imported, rather than being rebuilt for every request. If a non-empty Chroma directory already exists, the code reuses it. Consequently, changes to source documents are not automatically reflected in an existing index; the index should be rebuilt if the knowledge documents change.

Prototype versus production-oriented service: A notebook prototype is often used for experimentation by one developer, whereas an API allows other programs to submit requests over HTTP in a repeatable way. This experiment adds useful operational safeguards, but it is not a complete public production service. A deployed service would also need appropriate authentication, HTTPS, rate limiting, monitoring, secure configuration management, load testing, model evaluation, and a recovery plan.

CODE:

app.py — API Layer

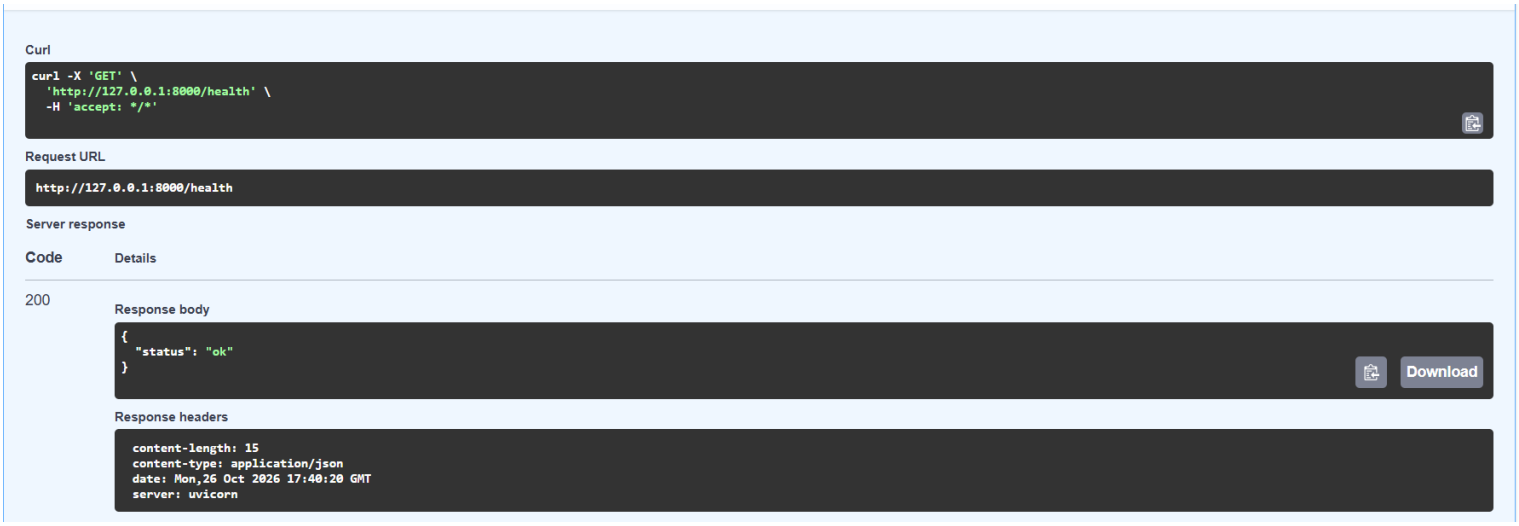
The app.py file creates the FastAPI application and defines two endpoints. GET /health returns {"status": "ok"}. POST /ask accepts a JSON request containing a question. The AskRequest model trims whitespace, rejects empty questions, and limits the question to 1,000 characters. The endpoint calls answer_question(), measures elapsed time, and returns the answer, sources, and response_time_ms. BackendError is converted into HTTP 502, while unexpected exceptions are logged and returned as a generic HTTP 500 response. Question text is deliberately not logged.

support_rag.py — Retrieval and Generation

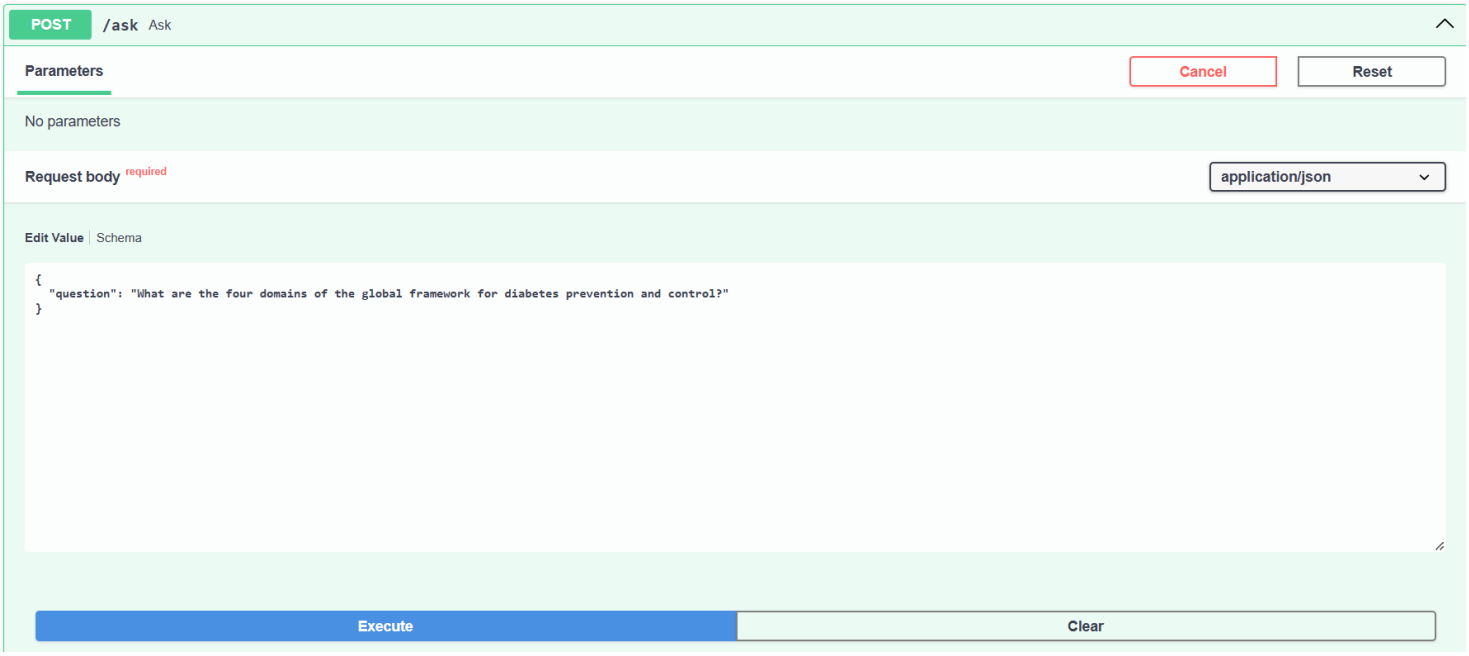
The support_rag.py file loads environment variables, defines the knowledge files and configuration, and initializes the embedding model and Chroma vector store. Documents are split into chunks when a new index is built. The retrieve() function returns the most similar chunks and their distances. answer_question() applies MAX_DISTANCE, constructs a prompt from the retrieved documents, calls Ollama through requests, and returns the generated answer with source filenames and page information. If retrieval is judged insufficient, it returns a fallback message instead of calling the LLM.

Experiments and Screenshots

Experiment 1: API documentation and health endpoint



Experiment 2: Supported question



Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/ask' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "question": "What are the four domains of the global framework for diabetes prevention and control?"
}'
```

Request URL

```
http://127.0.0.1:8000/ask
```

Server response

Code	Details
200	Response body

```
{
  "answer": "The four domains of the global framework for diabetes prevention and control are:\n\n1. Health system determinants\n2. Population health-related indicators\n3. Governance\n4. Financ
e",
  "sources": [
    "diabetes.pdf (page 20)",
    "diabetes.pdf (page 21)",
    "diabetes.pdf (page 4)"
  ],
  "response_time_ms": 2646.1
}
```

Download

Response headers

```
content-length: 307
content-type: application/json
date: Mon,26 Oct 2026 17:44:44 GMT
server: uvicorn
```

Responses

Experiment 3: Unsupported question

POST /ask Ask

Parameters Cance

No parameters

Request body required

Edit Value | Schema

```
{
  "question": "What is the capital of Japan?"
}
```

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/ask' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "question": "What is the capital of Japan?"
}'
```

Request URL

```
http://127.0.0.1:8000/ask
```

Server response

Code	Details
200	Response body

```
{
  "answer": "I don't have enough information in the provided knowledge base to answer that.",
  "sources": [],
  "response_time_ms": 66.7
}
```

Response headers

```
content-length: 128
content-type: application/json
date: Mon,26 Oct 2026 17:46:36 GMT
server: uvicorn
```

Responses

Code	Description

Experiment 4: Invalid input

POST /ask Ask

Parameters

No parameters

Request body required

Edit Value | Schema

```
{
  "question": "   "
}
```

```
curl -X 'POST' \
  'http://127.0.0.1:8000/ask' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "question": "   "
}'
```

Request URL

```
http://127.0.0.1:8000/ask
```

Server response

Code	Details
422	Error: Unprocessable Content

Response body

```
{
  "detail": [
    {
      "type": "value_error",
      "loc": [
        "body",
        "question"
      ],
      "msg": "Value error, Question must not be empty.",
      "input": "   ",
      "ctx": {
        "error": {}
      }
    }
  ]
}
```

Response headers

```
content-length: 143
content-type: application/json
date: Mon,26 Oct 2026 17:47:41 GMT
server: uvicorn
```

Responses

Experiment 5: Backend unavailable

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/ask' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "question": "What are the warning signs of dengue that require timely referral for specialized medical management?"
}'
```

Request URL

```
http://127.0.0.1:8000/ask
```

Server response

Code	Details

502
Undocumented
	Error: Bad Gateway
	Response body
	`{ "detail": "The language model service is currently unavailable." }`
	Response headers
	`content-length: 65`
	`content-type: application/json`
	`date: Mon,26 Oct 2026 17:53:46 GMT`
	`server: uvicorn`

Responses

Results and Observations

Test Case	Expected Result	Actual Result	Pass/Fail
Health endpoint	HTTP 200; response status is ok.	HTTP 200; {"status":"ok"} returned.	PASS
Supported question	Relevant answer grounded in retrieved sources, with response time.	HTTP 200; response included answer, source references, and response_time_ms (approximately 4543.466 ms recorded).	PASS
Unsupported question	Fallback message when retrieval is insufficient or distance exceeds threshold.	HTTP 200; fallback message: "I don’t have enough information in the provided knowledge base to answer that."	PASS
Empty question	HTTP 422 validation error for a blank question.	HTTP 422; validation error returned for an empty question.	PASS
Backend unavailable	HTTP 502 with a generic backend error message.	HTTP 502; response stated that the language model service is currently unavailable.	PASS
Response-time measurement	Successful /ask responses include response_time_ms and latency is logged.	API returned response_time_ms; application log recorded latency_ms=5388.1 ms.	PASS
Secrets protection	.env is excluded from version control and no secrets are exposed.	.env is excluded from Git, as verified by the user.	PASS

Conclusion

The application is structured as a FastAPI service around a RAG pipeline. It includes request validation, retrieval-based context selection, a local Ollama backend, controlled error responses, operational logging, and response-time measurement. The experiment is complete after the listed test cases have been executed and the actual results and screenshots have been recorded. This lab-level implementation is not a fully production-ready public service; authentication, HTTPS, rate limiting, monitoring, secure secret management, load testing, model evaluation, and a recovery plan would be additional requirements.

Discussion Questions

1. What is the purpose of FastAPI in this experiment?
FastAPI provides a web API interface for the Retrieval-Augmented Generation (RAG) application. It exposes endpoints such as /health and /ask, validates incoming requests, handles errors, and returns answers, source references, and response-time measurements.

2. What is the difference between a local LLM and a cloud-hosted LLM API?
A local LLM runs on the user's own computer or local server, providing greater control over data and reducing dependence on external services. A cloud-hosted LLM API runs on a remote provider's infrastructure and is accessed over a network. Cloud APIs may offer more computational resources but can involve usage costs, network latency, and data privacy considerations.

3. Does hosting FastAPI in the cloud automatically host the LLM there?

No. Hosting FastAPI in the cloud only deploys the API application. The LLM must be hosted separately or accessed through an external inference service. If the API configuration points to Ollama running on the developer's local machine, the deployed cloud API will not automatically have access to that local instance.

4. Why should an API key or secret not be hard-coded in Python source?
Hard-coded secrets can accidentally be exposed when source code is shared, committed to a public repository, or uploaded to version control. Environment variables and secret-management systems help keep credentials separate from application code and reduce the risk of unauthorized access.

5. What is input validation, and why is a maximum question length useful?
Input validation checks whether incoming data satisfies the application's requirements before processing. In this experiment, the application rejects blank questions and questions exceeding 1000 characters. A maximum length helps limit unnecessary processing, reduce resource consumption, and prevent excessively large requests.

6. What is the purpose and limitation of a health endpoint?
The /health endpoint indicates whether the FastAPI application is running and able to respond to requests. However, it does not necessarily verify that Ollama, the embedding model, the vector database, or other dependencies are working correctly.

7. Why should the model and vector index not be loaded for every request?
Loading models and vector indexes for every request increases response time and computational overhead. Initializing and reusing them when the application starts improves efficiency and reduces repeated loading costs.

8. What should RAG do when the answer is absent from its knowledge base?
RAG should avoid inventing an answer. It should return a fallback message when the retrieved documents are insufficiently relevant or when the similarity distance exceeds the configured threshold. This helps reduce unsupported responses.

9. Name two reasons an inference request might fail.
Two possible reasons are:
The Ollama backend is unavailable or unreachable.
The model takes too long to respond, causing a timeout.
Other possibilities include insufficient system resources or an invalid backend configuration.

10. Is this lab application fully production-ready? Name two additional requirements for production deployment.
No. The application demonstrates several production-oriented practices, including input validation, error handling, logging, and response-time measurement, but it is not necessarily fully production-ready.
Two additional requirements are:
Authentication and authorization: Protect API endpoints from unauthorized access.
HTTPS: Encrypt communication between clients and the API.
Other important requirements include rate limiting, monitoring, secure secret management, load testing, model evaluation, and a recovery plan.